

Module 15: Docker for Backend and DevOps Engineers

Questions

Q1. What is Docker, and how does it differ from a virtual machine?

Answer: Docker is a containerization platform that packages an application with its dependencies so it runs consistently across environments. A Docker container shares the host operating system kernel and runs as an isolated process using namespaces and cgroups. A virtual machine, in contrast, emulates an entire computer with its own guest operating system, kernel, and virtual hardware. Containers are lighter and start faster than VMs because they do not boot a full OS. VMs provide stronger isolation but consume more resources. For backend developers, Docker solves the it works on my machine problem. For DevOps engineers, Docker standardizes build, test, and deployment environments across laptops, CI/CD, and production.

Q2. What is the difference between a Docker image and a Docker container?

Answer: A Docker image is a read-only template that contains the application code, runtime, libraries, environment variables, and configuration needed to run an application. It is built from a Dockerfile and stored in layers. A Docker container is a running instance of an image. It adds a writable layer on top of the image and executes the process defined by CMD or ENTRYPOINT. Multiple containers can be created from the same image. Images are immutable and versioned, while containers are ephemeral by default and their local changes disappear when removed.

Q3. What happens when you run `docker run <image>`?

Answer: The Docker client sends the command to the Docker daemon. The daemon checks if the image exists locally and pulls it from the configured registry if it does not. It then creates a writable container layer over the image, sets up isolated namespaces for process, network, mounts, and other resources, assigns an IP address on the default bridge network, executes the command specified by CMD or ENTRYPOINT, and starts the main process as PID 1. If port mappings are specified with `-p`, the daemon configures iptables rules to forward traffic from the host port to the container port.

Q4. What are Docker namespaces and cgroups?

Answer: Namespaces provide process isolation by giving each container its own view of system resources. Docker uses PID namespaces for process isolation, NET namespaces for network interfaces, MNT namespaces for filesystem mounts, UTS namespaces for hostnames, IPC namespaces for shared memory, and USER namespaces for user and group IDs. Cgroups, or control groups, limit and account for resource usage such as CPU, memory, disk I/O, and network bandwidth. Namespaces keep containers separate, while cgroups prevent a single container from consuming all host resources. Together they make containers lightweight, isolated, and resource-controlled.

Q5. What is a Dockerfile, and what are the most common instructions?

Answer: A Dockerfile is a text file that defines how to build a Docker image. Common instructions include `FROM` to set the base image, `RUN` to execute commands during the build, `COPY` to copy files from the build context, `ADD` to copy and optionally extract archives or fetch URLs, `WORKDIR` to set the working directory, `ENV` to set environment variables, `ARG` to define build-time variables, `EXPOSE` to document ports, `CMD` to set the default command, `ENTRYPOINT` to set the main executable, and `USER` to switch to a non-root user. Best practice is to order instructions from least frequently changed to most frequently changed to maximize layer caching.

Q6. What is the difference between `CMD` and `ENTRYPOINT` in a Dockerfile?

Answer: `ENTRYPOINT` defines the executable that runs when a container starts, while `CMD` provides default arguments to that executable. If both are present, `CMD` arguments are appended to `ENTRYPOINT`. If the user passes arguments to `docker run`, they override `CMD` but are appended to `ENTRYPOINT` unless `--entrypoint` is used. A common pattern is `ENTRYPOINT ["java", "-jar"]` and `CMD ["app.jar"]`. Using the exec form, which is JSON syntax, avoids shell interpretation and allows proper signal handling. For backend services, `ENTRYPOINT` is useful when you want to enforce the runtime, and `CMD` is useful for providing default JVM options or profiles.

Q7. What is the difference between `COPY` and `ADD` in a Dockerfile?

Answer: `COPY` copies files or directories from the build context into the image. `ADD` does the same but can also automatically extract tar archives and fetch remote URLs. Because `ADD` has hidden behaviors, Docker best practice recommends using `COPY` for most cases to make the Dockerfile

predictable and easier to understand. `ADD` should only be used when you genuinely need local tar extraction or remote file download. For Java applications, `COPY` is the standard choice for JAR files and configuration.

Q8. How does Docker layer caching work, and why does instruction order matter?

Answer: Docker builds images in layers, and each instruction creates a new layer. If the build context for a layer has not changed, Docker reuses the cached layer instead of rebuilding it. Instruction order matters because once a layer changes, every layer after it must be rebuilt. To maximize caching, you should place instructions that change rarely at the top of the Dockerfile and instructions that change frequently at the bottom. For example, copy the Maven `pom.xml` and download dependencies before copying the rest of the source code, so dependency resolution is cached across builds.

Q9. What is a multi-stage build, and why is it useful?

Answer: A multi-stage build uses multiple `FROM` instructions in a single Dockerfile to separate the build environment from the runtime environment. For example, a Java application can be compiled and packaged in a Maven image, then only the resulting JAR is copied into a smaller runtime image such as Eclipse Temurin JRE or a distroless image. This reduces final image size, removes build tools and source code from production images, reduces attack surface, and improves deployment speed. Backend and DevOps teams use multi-stage builds to keep production images minimal and secure.

Q10. What are Docker volumes and bind mounts, and when do you use each?

Answer: A volume is a managed storage area managed by Docker, usually stored under `/var/lib/docker/volumes/`, and identified by a name. Volumes are portable, easy to back up, and work well for persistent data such as database files. A bind mount maps a specific host file or directory into the container using an absolute path. Bind mounts are useful for development when you want source code or configuration files on the host to reflect immediately inside the container. In production, named volumes are preferred because they do not depend on host path layout and are managed by Docker or orchestrators. Bind mounts can introduce host coupling and permission issues.

Q11. What is Docker Compose, and when should you use it?

Answer: Docker Compose is a tool for defining and running multi-container applications using a YAML file, typically `docker-compose.yml`. It allows you to define services, networks, volumes, environment variables, ports, and dependencies in a declarative format. Compose is ideal for local development, integration testing, and small multi-service deployments. For production orchestration at scale, Kubernetes or Docker Swarm is usually more appropriate. Backend developers use Compose to run an application alongside its database, cache, and message broker locally with a single command. DevOps engineers use it for CI pipelines and staging environments.

Q12. What is the difference between Docker Compose `ports` and `expose`?

Answer: `ports` maps a host port to a container port, making the service accessible from outside the Docker network, for example `8080:8080`. `expose` only documents that a container listens on a port and makes it available to other services on the same Docker network, but does not publish it to the host. In Compose, `expose` is mainly informational. For backend services, `ports` is used when developers or load balancers need to reach the container from the host, while `expose` is used for internal service-to-service communication.

Q13. What is the `depends_on` limitation in Docker Compose, and how do you handle readiness?

Answer: `depends_on` only guarantees container start order, not service readiness. A database container may start before the application, but the database may still be initializing when the application tries to connect. To handle readiness, you should configure the application to retry connections with exponential backoff, or use a startup probe wrapper script that waits for dependencies. Docker Compose also supports `condition: service_healthy` or `condition: service_completed_successfully` with health checks in newer versions. In production, readiness checks and retry logic are more reliable than startup ordering alone.

Q14. What are Docker bridge, host, and overlay networks?

Answer: Bridge is the default network driver that creates a private internal network on a single host and allows containers to communicate using DNS names or IP addresses. Host mode removes network isolation and uses the host network stack directly, which can improve performance but reduces isolation. Overlay networks connect containers across multiple Docker hosts, commonly used in Docker Swarm or overlay plugins. For backend development on a single machine, bridge networks provide

sufficient isolation and service discovery. In production multi-host clusters, overlay or Kubernetes networking is used.

Q15. How do you persist data for a database running in a Docker container?

Answer: You use a named volume or bind mount to store database files outside the container's writable layer. For example, a PostgreSQL container maps `/var/lib/postgresql/data` to a named volume. If the container is removed, the data remains in the volume and a new container can mount it. You should also set the correct ownership and permissions for the database user, especially when using bind mounts. Backing up volumes and testing restore procedures is essential for production. For DevOps, this means including volume backup and disaster recovery in the operational runbook.

Q16. How do you run a Docker container as a non-root user?

Answer: You create a dedicated user in the Dockerfile using `RUN groupadd` and `useradd`, switch to that user with `USER`, and ensure files and directories the application needs are writable by that user. Running as root inside a container increases security risk because an attacker who escapes the container gains root privileges on the host. For Java applications, the base image may already provide a non-root user such as `appuser`, or you can add one. You also verify that port binding does not require root by using non-privileged ports such as 8080 instead of 80.

Q17. What are Docker health checks, and how do you configure them?

Answer: A health check is a command Docker runs periodically to determine if a container is healthy. You can define it in the Dockerfile with `HEALTHCHECK` or in Docker Compose with the `healthcheck` section. For a Spring Boot application, a common health check is `curl -f http://localhost:8080/actuator/health`. Docker marks the container as healthy, starting, or unhealthy based on the exit code. Orchestrators use health status for readiness probes and restart decisions. Health checks should be lightweight and should not overload the application or dependencies.

Q18. What is the difference between `docker stop` and `docker kill`?

Answer: `docker stop` sends a `SIGTERM` signal to the main process and waits for a grace period, default 10 seconds, before sending `SIGKILL`. This allows the application to shut down gracefully by closing connections, flushing buffers, and completing in-flight requests. `docker kill` sends `SIGKILL` immediately, forcing the process to terminate without cleanup. For backend services, graceful shutdown is important to avoid data loss and incomplete requests. The Dockerfile should use the exec form of ENTRYPOINT so the application receives signals directly as PID 1.

Q19. How do you pass secrets to a Docker container securely?

Answer: You should avoid hard-coding secrets in images or environment variables in plain text. Secure options include using Docker secrets if running on Docker Swarm, mounting secrets through BuildKit `--secret` during build, using environment files with restricted permissions at runtime, or integrating with external secret managers such as HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault. In Kubernetes, secrets are stored as Secret resources and mounted as files or environment variables. The principle is to keep secrets out of image layers and build logs, rotate them regularly, and limit access to authorized users and processes.

Q20. What is the difference between Docker Swarm and Kubernetes?

Answer: Docker Swarm is Docker's native clustering and orchestration tool. It is simpler to set up and uses Docker Compose files for service definitions. Kubernetes is a more powerful and widely adopted orchestration platform with a richer feature set for service discovery, scaling, self-healing, networking, storage, and rolling updates. Kubernetes has a steeper learning curve but is the de facto standard for production container orchestration. For small teams or simple deployments, Swarm may be sufficient. For enterprise microservices, Kubernetes is usually preferred. Both can manage multi-container applications, but Kubernetes offers more ecosystem support and vendor integration.

Q21. What is `.dockerignore`, and why is it important?

Answer: `.dockerignore` is a file that tells Docker which files and directories to exclude from the build context. Excluding files such as `target/`, `.git/`, IDE directories, local configuration, and sensitive files reduces the build context size, speeds up image builds, and prevents accidental inclusion of secrets or unnecessary files in the image. It also improves cache efficiency because changes to ignored files do not invalidate build cache. Backend developers should keep `.dockerignore` up to date with build artifacts and source-control metadata.

Q22. How do you reduce the size of a Docker image?

Answer: I use a smaller base image such as Alpine Linux, Distroless, or Eclipse Temurin JRE instead of a full JDK. I use multi-stage builds to keep only the compiled artifact in the final image. I minimize the number of layers by combining related RUN commands. I clean package caches within the same RUN instruction. I use `.dockerignore` to avoid copying unnecessary files. I remove build tools, compilers, and shells from production images when possible. I also avoid installing recommended packages unless needed. Smaller images reduce attack surface, startup time, storage cost, and network transfer time during deployments.

Q23. What are container restart policies, and when should you use each?

Answer: Docker supports restart policies that define what happens when a container exits. `no` means the container does not restart automatically. `always` restarts the container regardless of exit code. `unless-stopped` restarts unless the container was manually stopped. `on-failure` restarts only if the container exited with a non-zero code, optionally limited to a maximum retry count. For backend services, `unless-stopped` or `on-failure` are common choices. `on-failure` is useful when you want to retry after application crashes but not after a clean shutdown. Restart policies help improve availability but should be combined with proper logging and alerting.

Q24. How do you build a Docker image from a Spring Boot application?

Answer: You create a Dockerfile in the project root. A typical approach uses a multi-stage build. The first stage uses a Maven or Gradle image to build the application and produce a JAR file. The second stage copies the JAR into a lightweight JRE image such as `eclipse-temurin:17-jre-alpine` and sets the entrypoint to run the JAR. Alternatively, Spring Boot 2.3 and later support Cloud Native Buildpacks through the Maven `spring-boot:build-image` goal, which builds an optimized image without writing a Dockerfile. You can also use the `dockerfile-maven-plugin` or `jib-maven-plugin` to build and push images from Maven.

Scenario based questions

S1. A Spring Boot container builds successfully but exits immediately with exit code 1. How do you debug it?

Answer: I run `docker ps -a` to find the exited container and then `docker logs <container-id>` to read the application output. Exit code 1 usually indicates the application failed to start, often

because of a missing database connection, invalid configuration, or a missing profile. I check whether environment variables expected by Spring Boot are passed correctly, whether the JAR path in ENTRYPOINT is correct, and whether the active profile matches the running environment. I also run the container interactively with `docker run -it --rm --entrypoint sh <image>` to inspect files and run the JAR manually if needed. Real-time example: a developer builds an order service image and runs it in a Compose stack. The container exits immediately. Logs show `Failed to configure a DataSource: 'url' attribute is not specified`. The `SPRING_DATASOURCE_URL` environment variable was defined in a `.env` file that was not loaded because the file was named `.env.local`. I rename the file, verify with `docker compose config`, and the container starts successfully.

S2. A container starts but the application is unreachable on the mapped host port. What do you check?

Answer: I first verify that the port mapping was specified correctly in `docker run` or Compose, for example `-p 8080:8080`. I check `docker ps` to see if the port is actually published. I confirm the application inside the container is listening on `0.0.0.0` and not only `127.0.0.1`, because binding to localhost inside a container makes the port unreachable from the host. I check firewall rules, SELinux, and whether another process is already using the host port. I also look at application logs for startup errors and use `docker exec` to run `netstat` or `curl` inside the container. Real-time example: a Node.js service binds to `127.0.0.1:3000` and the Dockerfile exposes port 3000. The container runs but `curl http://localhost:3000` from the host returns connection refused. I change the application to bind to `0.0.0.0` and the port becomes reachable.

S3. A Docker build fails with `COPY failed: file not found in build context`. What is the cause?

Answer: The `COPY` instruction references a file or directory that does not exist in the build context, or it is excluded by `.dockerignore`. I verify the path is relative to the build context root, not the Dockerfile directory if they differ. I check that the file was generated before the build, especially JAR files in `target/` or `build/libs/`. I also inspect `.dockerignore` to make sure the file is not ignored. If the file is generated by a previous build step, I ensure the build command includes the correct context path. Real-time example: a CI pipeline runs `docker build -t app .` without first running `mvn package`. The Dockerfile copies `target/app.jar`, but the directory is empty. I add `mvn package` before the Docker build step and the build succeeds.

S4. A Java application container exits with exit code 137. What does it mean, and how do you fix it?

Answer: Exit code 137 means the container received SIGKILL, almost always because it exceeded its memory limit and was Out-Of-Memory killed by the kernel. I check `docker inspect` for OOMKilled status and review memory settings with `docker stats`. To fix it, I either increase the memory limit or reduce the application's heap size. For JVM applications, I set `-Xmx` to a value well below the container memory limit to leave room for off-heap memory, native libraries, and the JVM itself. I also investigate memory leaks if usage grows continuously. Real-time example: a Spring Boot container is limited to 512 MB but the JVM uses `-Xmx1g`. The container is OOM-killed under load. I change the Dockerfile to use `-Xmx384m` and raise the container limit to 768 MB, then verify with load testing that the application stays stable.

S5. A container cannot resolve the hostname of another container in Docker Compose. What do you check?

Answer: I check whether both containers are on the same custom network. Containers on the default bridge network cannot resolve each other by name; only custom networks provide embedded DNS. In Docker Compose, all services in the same Compose project are automatically connected to a default custom network, so the issue is usually a typo in the service name or using `localhost` instead of the service name. I verify the service name in `docker-compose.yml`, check that both services are running with `docker compose ps`, and test resolution with `docker exec` and `nslookup` or `ping`. Real-time example: an application service connects to `localhost:5432` expecting the Postgres container. Because each container has its own localhost, the connection fails. I change the connection string to `postgres:5432`, which resolves through the Compose network, and the application connects successfully.

S6. A Docker Compose service depends on a database, but the application crashes before the database is ready. How do you fix it?

Answer: I implement application-level retry logic for database connections instead of relying solely on `depends_on`. I configure the connection pool to retry with exponential backoff during startup, or use a small wait-for-it script that checks database readiness before starting the application. In newer Docker Compose versions, I use `depends_on` with `condition: service_healthy` and define a health check on the database service. The health check ensures the database is truly accepting connections before the dependent service starts. I avoid naive `sleep` scripts because they make startup slow and unreliable. Real-time example: a Spring Boot application starts before Postgres

finishes initialization. I add a `healthcheck` to the Postgres service using `pg_isready -U appuser -d orders`, set the app service `depends_on` with `condition: service_healthy`, and configure the connection pool with a 30-second acquisition retry. The stack now starts reliably.

S7. A Docker image contains a large layer from `COPY . .` because of unnecessary files. How do you optimize it?

Answer: I create or update `.dockerignore` to exclude files that are not needed in the image. This includes build output directories like `target/` and `build/`, version control directories like `.git/`, IDE files like `.idea/`, local environment files, test resources, and logs. Excluding these reduces the build context, speeds up the build, and prevents sensitive or irrelevant files from being copied. I also order Dockerfile instructions so that dependency resolution happens before source code changes, maximizing layer caching. Real-time example: a 2 GB build context slows every build because it includes a local `.git` directory and test data files. After adding those paths to `.dockerignore`, the build context drops to 30 MB, build time decreases by 80 percent, and the final image no longer contains source code.

S8. A production container needs to read secrets. Why should you not use environment variables, and what do you use instead?

Answer: Environment variables are often logged by orchestrators, debugging tools, and crash dumps. They can be exposed through `docker inspect`, process listings, or CI logs. Instead, I use Docker secrets for Swarm, Kubernetes Secrets, or a secret manager that mounts secrets as files into the container. For build-time secrets, I use BuildKit secret mounts so secrets never become part of the image layer. At runtime, the application reads secrets from files in a mounted directory or fetches them from Vault or a cloud secret manager. This keeps secrets out of image layers, process environments, and version control. Real-time example: a developer passes `DATABASE_PASSWORD=secret123` in the Compose file and checks it into Git. I replace it with a Docker secret mounted at `/run/secrets/db_password`, update the application to read from the file, remove the variable from the Compose file, and rotate the leaked password immediately.

S9. A multi-stage Dockerfile copies dependencies in every build even when they did not change. How do you fix it?

Answer: I copy only the dependency descriptor files first, resolve dependencies, then copy the source code. For Maven, I copy `pom.xml` and any parent POMs, run `mvn dependency:go-offline` or

`mvn install -DskipTests`, and then copy the source. For Gradle, I copy `build.gradle`, `settings.gradle`, and the Gradle wrapper first. Because Docker caches layers based on file changes, dependency resolution is cached until the descriptor files change. Source code changes only invalidate the later layers. Real-time example: a Dockerfile starts with `COPY . .` and runs `mvn package`. Every code change re-downloads all Maven dependencies. I restructure the Dockerfile to copy `pom.xml` first, run `mvn dependency:resolve`, then copy `src/`. Builds now reuse the dependency layer and complete in seconds after small code changes.

S10. A Spring Boot container logs to a file inside the container, but logs are lost when the container restarts. How do you persist them?

Answer: I map the log directory to a named volume or bind mount on the host so logs survive container restarts. Better yet, I configure the application to log to standard output and let Docker capture the logs, then centralize logs with a log shipper such as Filebeat, Fluentd, or the container orchestrator's logging driver. Logging to stdout is the Twelve-Factor App recommendation and simplifies log collection, rotation, and aggregation. If file logging is required, I ensure the volume is backed up and rotated to avoid filling the disk. Real-time example: an application writes logs to `/app/logs/application.log`. After a crash, the container restarts and the logs are gone, making it impossible to diagnose. I change the logging configuration to output JSON to stdout, configure Docker's `json-file` log driver with rotation, and ship logs to Elasticsearch with Filebeat. Logs are now searchable and retained.

S11. A container running as non-root cannot write to a bind-mounted directory. Why?

Answer: Bind mounts preserve the user and group ownership from the host. If the container user has a different user ID or group ID than the owner of the host directory, write permission is denied. I fix this by ensuring the container user ID matches the host directory owner, by setting appropriate host permissions, or by using a named volume which is managed by Docker and usually owned by the container user. In development, matching UIDs is common. In production, named volumes are preferred to avoid host coupling. For Linux, SELinux may also block access unless the mount is relabeled with `:z` or `:Z`. Real-time example: a Spring Boot container runs as UID 1000 but the host bind mount `/opt/data` is owned by UID 1001. The application fails to write uploaded files. I create a host user with UID 1000, chown the directory, and the writes succeed.

S12. A Docker image size is over 1 GB for a simple Spring Boot application. How do you reduce it?

Answer: I switch from a full JDK base image to a JRE or distroless image. I use a multi-stage build so only the JAR and runtime dependencies remain in the final image. I remove build tools, shells, and package managers from the production image. I combine RUN commands to reduce layers and clean package caches in the same layer they are created. I also use `.dockerignore` to avoid copying unnecessary files. For example, replacing `eclipse-temurin:17-jdk` with `eclipse-temurin:17-jre-alpine` and using a distroless final stage can reduce a 1 GB image to under 200 MB. Real-time example: a team ships a 1.2 GB image that includes Maven, Gradle, source code, and a full JDK. I restructure the Dockerfile to build in a Maven stage and copy the JAR into `eclipse-temurin:17-jre-alpine`. The final image shrinks to 180 MB, deployment time drops, and the attack surface is reduced.

S13. A containerized application does not shut down gracefully when `docker stop` is issued. How do you fix it?

Answer: I ensure the main application process runs as PID 1 and receives signals directly. This means using the exec form of ENTRYPOINT such as `ENTRYPOINT ["java", "-jar", "app.jar"]` instead of the shell form, which runs the command as a child of a shell process that may not forward signals. I configure the application to handle SIGTERM by closing resources, draining connections, and stopping the server. I also verify that the stop timeout is sufficient for graceful shutdown. Real-time example: a Spring Boot container uses shell-form ENTRYPOINT `java -jar app.jar`. On `docker stop`, the shell ignores SIGTERM and Docker waits 10 seconds before sending SIGKILL, causing in-flight requests to abort. I change to exec form, add graceful shutdown in Spring Boot with `server.shutdown=graceful`, and extend the stop grace period to 30 seconds. The application now finishes active requests before exiting.

S14. A Docker Compose stack works on one machine but fails on another. How do you troubleshoot?

Answer: I compare Docker versions, Compose file syntax versions, and platform differences such as Linux versus Windows containers. I check for environment variables or `.env` files that differ between machines. I verify that the other machine has sufficient resources, including memory, disk space, and CPU. I inspect for port conflicts where a service tries to bind to a host port already in use. I also check for differences in file permissions, line endings if using bind mounts, and the presence of required secrets or certificates. Running `docker compose config` expands the file and reveals substitution

and parsing issues. Real-time example: a stack runs on a developer's Mac but fails on a CI runner. The CI runner has an older Docker Compose version that does not support the `profiles` syntax used in the file. I either update the runner or remove the unsupported syntax, and the stack starts consistently.

S15. A microservice architecture with five services needs to run locally for development. How do you Dockerize it?

Answer: I create a `docker-compose.yml` that defines each service with its build context or image, ports, environment variables, and dependencies. Each service has its own Dockerfile, typically a multi-stage build producing a small production-ready image. I add a shared custom network so services can communicate by name. I include supporting infrastructure such as a database, message broker, and cache as services in the Compose file or as named volumes for data persistence. I use `.env` files for environment-specific configuration and avoid hard-coding secrets. I define health checks and readiness dependencies so the stack starts reliably. Real-time example: an e-commerce platform has gateway, order, payment, inventory, and notification services plus Postgres, Kafka, and Redis. I create a Compose file that builds each service from its Dockerfile, exposes the gateway on port 8080, connects everything to a `backend` network, and uses health checks to ensure Postgres and Kafka are ready before services start. Developers run `docker compose up` and the entire stack is available locally.

S16. A container registry scan reports a critical vulnerability in a base image. How do you respond?

Answer: I first verify the severity and whether the vulnerable component is actually used by the application. If the vulnerability affects a library or package used by the runtime, I update the base image to the latest patched version. I rebuild the application image, run tests, and push the new image to the registry. I also update the CI pipeline to use the patched base image and schedule regular scans. For false positives, I document the exception. If the vulnerable package is unnecessary, I remove it or use a minimal distroless image to reduce attack surface. Real-time example: a scan finds a critical OpenSSL vulnerability in the base image. I update `FROM eclipse-temurin:17-jre-alpine` to the latest digest, rebuild all dependent images, deploy through CI/CD, and configure the registry to block future builds that contain the old base image digest.

S17. A `docker push` to a private registry fails with authentication error. What do you check?

Answer: I check that I am logged in to the registry using `docker login` with valid credentials. I verify the registry URL in the image tag matches the login target. I check whether the token or password has

expired, especially for cloud registries that use short-lived tokens. I review permissions to ensure the user or service principal has push access to the repository. If running in CI, I verify that the credentials are correctly passed as secrets and not masked incorrectly. For Amazon ECR, I also check that the token was regenerated before the push because ECR tokens expire after 12 hours. Real-time example: a CI pipeline pushes to a GitHub Container Registry and fails with `denied: requested access to the resource is denied`. The personal access token used in `docker login` was revoked. I generate a new token, update the CI secret, and the push succeeds.

S18. A container is consuming too much CPU and slowing down the host. How do you investigate and limit it?

Answer: I use `docker stats` to identify the container with high CPU usage. I inspect the application logs and metrics to find the cause, such as an infinite loop, inefficient algorithm, or unexpected load. I limit CPU usage with `--cpus` or `deploy.resources.limits.cpus` in Compose to prevent the container from starving other services. For long-term fixes, I optimize the application code, add caching, or scale horizontally. I also set memory limits to prevent CPU issues from turning into OOM kills. Real-time example: a container running a PDF generation service spikes to 400 percent CPU and affects other services. I add `--cpus=2` to cap it, add metrics to identify the slow PDF operation, and refactor the code to stream processing. CPU usage stabilizes and other services recover.

S19. A database container with a named volume needs to be moved to a new host. How do you migrate the data?

Answer: I stop the container, create a backup of the volume using `docker run --rm -v myvolume:/data -v $(pwd):/backup alpine tar czf /backup/volume.tar.gz -C /data .`, transfer the archive to the new host, and restore it with `docker run --rm -v myvolume:/data -v $(pwd):/backup alpine sh -c "cd /data && tar xzf /backup/volume.tar.gz"`. I then start the new container with the restored named volume. For large databases, I prefer database-native backup tools such as `pg_dump` or `mysqldump` because they are more reliable and consistent. I also verify the restored data by running health checks and a few application queries. Real-time example: a Postgres container on a developer laptop needs to move to a staging server. I use `pg_dump` to create a consistent dump, copy it to the server, create the new volume, restore with `pg_restore`, and start the Compose stack. The application starts and data is intact.

S20. A team wants zero-downtime deployment for a containerized Spring Boot service. How do you design it?

Answer: For Docker Compose or single-host setups, I use rolling updates by starting new containers behind a reverse proxy or load balancer, waiting for health checks to pass, and then stopping old containers. For orchestrated environments, I use Kubernetes rolling updates with readiness probes and graceful termination. The key requirements are multiple running instances, a load balancer that routes only to healthy instances, and proper SIGTERM handling so existing requests complete. I also use blue-green deployments for critical services by switching traffic between two identical environments. Image tags should be immutable, such as git commit SHA or semantic version, not `latest`. Real-time example: a payment service runs as a Kubernetes Deployment with three replicas. I release a new version by updating the Deployment image tag. Kubernetes creates new pods, waits for readiness probes to succeed, gradually shifts traffic, and terminates old pods with a 30-second grace period. No requests are dropped because the application handles SIGTERM gracefully and Kubernetes only removes pods after the readiness endpoint marks new pods healthy.

S21. A container needs to access host services such as a local database. What networking options are available?

Answer: On Docker Desktop, `host.docker.internal` resolves to the host from inside a container. On Linux, this requires extra configuration or using the host network mode with `--network host`, which removes container network isolation. Another option is to connect the container and the host service to a common bridge network if the host service also runs in a container. For production, services should not depend on the host network. For local development, `host.docker.internal` is the most portable approach when supported. I prefer to run dependencies in containers and use Compose networks rather than relying on the host. Real-time example: a developer runs a local Postgres on the laptop and wants a containerized service to connect to it. On Docker Desktop, I configure the service connection string to use `host.docker.internal:5432` and the application connects successfully. On a Linux CI runner, I instead add a Postgres service container to the same network as the application.